LangChain: Human Approval Across Resume

Grounded DI LLC | October 2026 | Public research edition

A configured approval gate can pause a tool call, but a conditional review predicate must remain consistent when execution resumes. This study demonstrates the documented consequence of deliberately changing that predicate, alongside controls where it remained stable.

Evidence at a glance

Documented design: configured tool calls are interrupted for review; batched responses are positional. Source explicitly warns that a predicate returning false on resume can cause the reviewer answer to be skipped and the tool to run. This is a stated configuration condition, not a newly established vendor defect.

Observed local result: in both batched and per-call modes, a rejection was supplied after the predicate changed from true to false; the in-memory tool recorder then recorded alpha. Stable-predicate rejection controls recorded no tool call. Approval controls recorded one call.

This is a bounded local experiment with an intentionally unstable predicate. It does not establish an unauthorized actor, a hosted deployment failure, customer incident, or production bypass. No live service or external effect was tested.

Scope, source and method

Pinned source

LangChain commit 399fd2e433e54cefdbddb3ede1079ddd3f8ca9c4; LangChain Python subtree Git tree ae87cd9b0f8fc1c1cac4a8a3d74f60699ea02ee5; middleware Git blob 161fc0d6bd662114e07ae1bfdc7fc121d1031edd. The retrieved subtree archive SHA-256 begins 18552731742e352f and is recorded in full in the verification record.

The follow-up imports the pinned LangChain source subtree. LangGraph, LangChain Core and other dependencies are separately installed versions; they are not pinned to the same repository commit. Dependency versions were recorded, but package artifact hashes were not captured. The LangChain source archive does not include a pinned LangGraph source tree.

Method and observation

A fake model requests a harmless in-memory write_note call. An in-memory checkpointer pauses execution; the script supplies an approval or rejection on resume. The tool appends its input to a local list. The follow-up records call ID and a canonical argument digest at predicate evaluation and tool invocation. This is script-generated instrumentation, not an independent witness of execution.

The rejected answer is an input supplied by the script. When the predicate flips, the record does not establish that middleware consumed the rejection; source indicates that the answer is skipped. Earlier language describing a processed rejection was corrected.

Results and interpretation

Follow-up cases

Six registered cases matched expected outcomes. Batched mode: stable approval recorded alpha; stable rejection recorded no call; rejection supplied after a true-to-false predicate change recorded alpha. Per-call mode showed the same three outcomes. The predicate-change scenarios deliberately violate the source-described stability requirement.

Two separately registered batch-order controls were repeated using the pinned subtree: approve alpha/reject beta recorded alpha; reject alpha/approve beta recorded beta. The repeated stable-predicate rejection recorded no call; the deliberately changed predicate recorded alpha. Four of four matched their expected results.

What the controls establish

The local stable cases demonstrate that approval and rejection behaved as expected for this fixture. The batch cases demonstrate positional decisions in the tested configuration. They do not show how a real reviewer, durable checkpointer, application authorization policy, or external system behaves.

The test harness generated both the resume input and observation. Call ID and argument digest improve correlation within the fixture; they do not independently authenticate the reviewer or verify an external effect. No independent certification is claimed.

Application boundary and next tests

An application relying on a conditional interrupt should ensure that its review predicate gives the same answer for the same paused call on resume. It should also preserve decision order in batched mode and protect the state and identities used for approval. These are engineering considerations informed by the documented contract and the local demonstrations.

A next authorized test would pin LangGraph and dependencies as well as LangChain, use a durable checkpointer and separately observed synthetic target, capture the approval request, resume decision, call identity and effect, and explore process restarts and predicate state changes. It would remain distinct from a hosted-service assessment.

Source ledger

Pinned middleware: https://github.com/langchain-ai/langchain/blob/399fd2e433e54cefdbddb3ede1079ddd3f8ca9c4/libs/langchain_v1/langchain/agents/middleware/human_in_the_loop.py

LangChain human review documentation: https://docs.langchain.com/oss/python/langchain/human-in-the-loop

LangGraph interrupts: https://docs.langchain.com/oss/python/langgraph/interrupts

The earlier six direct middleware checks and three single-call graph checks were run in a mixed installed environment. The DI2 native baseline is a separate harness run and is not a LangChain test. No earlier receipt was overwritten.